



Iby and Aladar Fleischman
Faculty of Engineering
Tel Aviv University

הפקולטה להנדסה
ע"ש איבי ואלדר פליישר
אוניברסיטת תל-אביב

The School of Electrical Engineering

בי"ס להנדסת חשמל

Project Work Plan

Project Number :3408

Project Name: התאמת קצב השיר למדדים הפיזיולוגיים בגוף

Students:

Name: Matan Granot ID 211696406

Name: Yogev Grinblatt ID 212365787

Project carried out at: _Tel Aviv University

For the project instructor:

I approve the submission of the following report

Signature: Dr. Lior Arbel

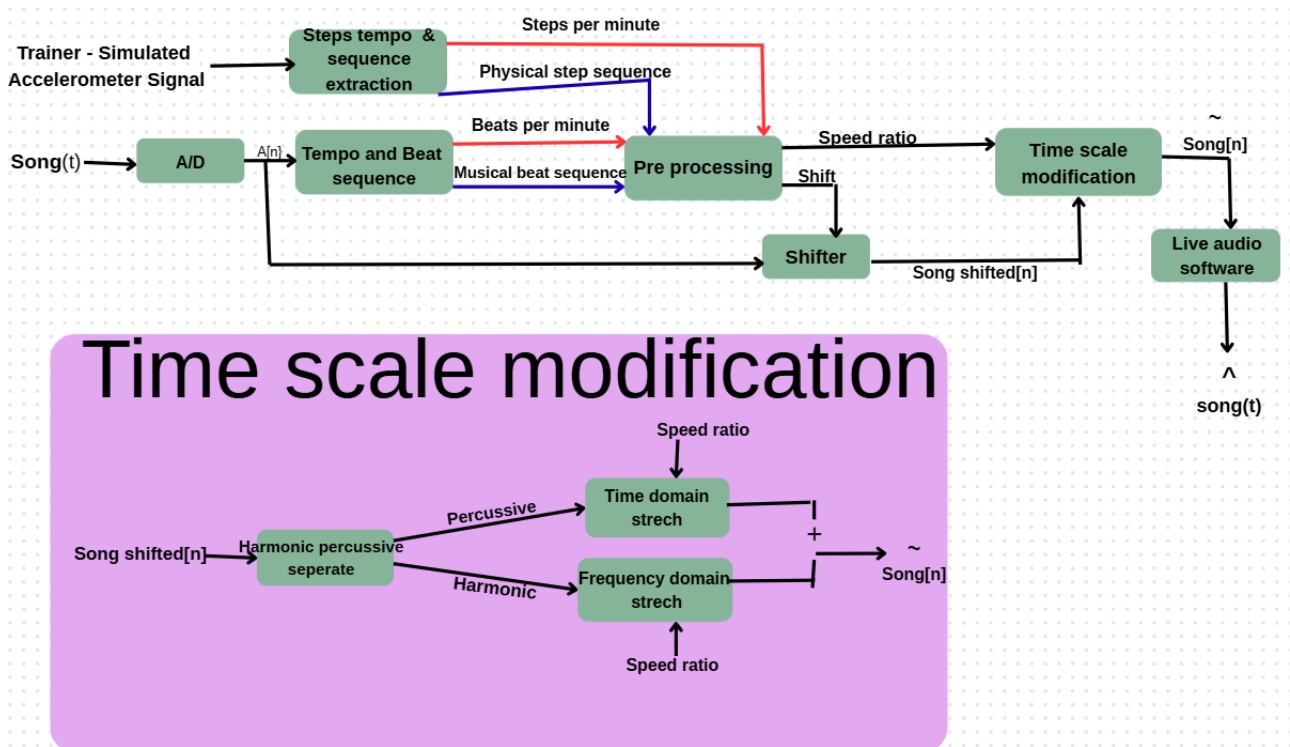
Name: Lior Arbel

1 Abstract

This project focuses on dynamically adjusting the playback rate of music to match a runner's physical rhythm in real time, using advanced musical digital signal processing (DSP) techniques. The core idea is to analyse both the runner's step frequency and the tempo of the selected music track, then calculate an appropriate speed ratio that allows the song to synchronize with the runner's movements. The runner's rhythm will be simulated by simulation of accelerometer signal (generated from python code or free web resources), while the song's rhythm is pre-processed to extract its beats and tempo. This speed ratio is then applied to the music stream using time-scale modification (TSM) algorithms, which can alter the playback speed without changing the pitch. The result is a seamless experience where the music adapts in real time to the runner's pace, enhancing motivation and coordination.

The project belongs to the field of musical digital signal processing, a branch of DSP that focuses on the analysis, synthesis, and transformation of audio signals. Musical DSP involves techniques for beat detection, tempo estimation, and real-time audio modification, all of which are central to this project. By combining rhythm extraction, tempo computation, and time-scale modification, the project applies foundational DSP concepts to a practical, interactive application.

The technologies developed in this project are widely applicable across multiple domains. In sports and fitness, adaptive music can enhance training by matching the tempo of audio to the user's activity level. In entertainment, similar techniques are used in DJing and live music performance to synchronize tracks seamlessly. Moreover, applications in accessibility and rehabilitation benefit from rhythm-aligned audio, helping users maintain or recover motor coordination. Beyond these, musical DSP technologies also underpin commercial streaming services, interactive media, and even virtual reality experiences where real-time audio adaptation improves immersion and responsiveness.



2 Motivation

The motivation for this project arises from the increasing demand for adaptive and interactive audio systems that enhance human performance and experience through real-time synchronization of music and movement. Traditional music playback applications provide a fixed tempo, which cannot adapt to the dynamic pace of a runner or athlete. This limitation reduces the potential benefits of rhythm-aligned audio, such as improved coordination, motivation, and training efficiency. By developing a system capable of adjusting music tempo live to match a user's physical rhythm, this project addresses a significant gap in the intersection of musical digital signal processing (DSP) and real-time interactive applications.

From a scientific and engineering perspective, this project leverages fundamental DSP concepts such as beat detection, tempo estimation, and time-scale modification (TSM). These techniques allow for accurate extraction of rhythmic information from both the user's movements and pre-recorded music, followed by precise and pitch-preserving adjustment of the audio stream. The resulting system contributes to the field by demonstrating a practical implementation of real-time DSP algorithms in an interactive environment, bridging the gap between theoretical research in rhythm analysis and applied audio engineering.

Compared to existing alternatives, the advantages of this approach are evident. Conventional music apps that offer pre-set tempo playlists or manual speed adjustment (e.g., fitness music apps like Spotify Running) provide limited adaptability and do not respond dynamically to the runner's instantaneous pace. Another alternative is metronome-based synchronization, where the user attempts to follow a fixed rhythmic cue, which lacks the flexibility and natural alignment that live audio adjustment can provide. In contrast, the proposed system continuously monitors the user's rhythm and applies DSP-based tempo modification, ensuring seamless and real-time synchronization that improves the training or entertainment experience.

In summary, this project addresses the limitations of fixed-tempo music playback and non-adaptive rhythmic cues by implementing an interactive, DSP-driven solution. It contributes to both scientific understanding and practical engineering applications of musical signal processing, with potential impacts in sports, rehabilitation, entertainment, and other domains requiring rhythm-aware audio systems.

3 Statement of Work

Project Requirements

The main requirement of this project is to design and implement a software system capable of adjusting the playback rate of a music track in real time to match a runner's rhythm. This involves several sub-requirements:

1. **Runner Rhythm Extraction:** Initially, the system will use **simulated runner signals** to simulate signal true running data, we will **create signals over time that simulate cadence of a runner**. If time permits, the system may be extended to accept live data from ready made **accelerometer** device. The runner signal will be correlated to the music signal to compute the speed ratio.
2. **Music Rhythm Analysis:** Preprocessing of the music track includes extracting the tempo (BPM) and, optionally, the beat sequence. The preprocessor receives both the runner signal and the music signal, then determines the **speed ratio**. Two approaches will be examined:
 - Using **only BPM** for coarse tempo matching.
 - Using **BPM and beat positions** to calculate the **best speed ratio and phase shift** to optimize alignment between runner steps and music beats.
3. **Speed Ratio Computation:** The core algorithm computes the ratio between runner rhythm and music rhythm based on the preprocessing step, either using average BPM or incorporating beat alignment.
4. **Time-Scale Modification (TSM):** The TSM module will implement two algorithmic approaches:
 - **Phase Vocoder:** operates in the frequency domain, preserving pitch while allowing tempo changes.
 - **WSOLA (Waveform Similarity Overlap Add):** operates in the time domain, providing more natural-sounding real-time modifications for small frame adjustments.

The focus is on **building the TSM module from scratch**, rather than relying solely on existing libraries, to gain practical understanding of DSP implementation.

5. **Simulation and Evaluation:** The project includes a set of controlled simulations using synthetic runner signals to validate:
 - BPM-only synchronization: adjusting music speed based on average tempo.
 - Beat-based synchronization: adjusting music speed and applying a phase shift to align beats with runner steps.
6. **Offline to Online Adaptation:** While the audio system itself will be non-intelligent (simply outputting the processed audio stream), the algorithms will be designed to **operate in an online fashion**: the runner's rhythm can be continuously analyzed, and the speed ratio can be updated dynamically. This involves slicing the music track into frames corresponding to the current speed ratio and ensuring smooth TSM application. The algorithm will examine **optimal frame size** for live playback to minimize artifacts while maintaining tempo alignment.

Tools, Environment, and Hardware

- **Programming Environment:** Python, for algorithm development and simulation. Key libraries include `numpy` and `scipy` for numerical and signal processing operations, `librosa` for offline music analysis, and `matplotlib` for visualization.
- **TSM Implementation:** Custom implementation of **WSOLA** and **phase vocoder** algorithms. Existing TSM libraries (e.g., Rubber Band) may be examined for comparison but not directly used in the final system.
- **Audio Streaming:** `sounddevice` or `pyaudio` for audio output during live simulation. The system will process frames according to the computed speed ratio.
- **Hardware:** Initially, no hardware is required beyond a standard PC. If live runner data is incorporated, an **accelerometer** (Arduino-based or commercial wearable) will provide step signals.

Mathematical and Algorithmic Theory

1. **Runner Rhythm Extraction:** Step signal $r(t)$ is correlated with music beats b_j using moving average or cross-correlation to estimate the best speed ratio.
2. **Music Rhythm Extraction:** From the audio signal $x[n]$, the tempo and beat positions are computed using onset detection and frame analysis as described in Müller (2015).
3. **Speed Ratio Computation:** The speed ratio r applied to TSM is:

$$r = \frac{BPM_{\text{runner}}}{BPM_{\text{song}}}$$

In beat-aware mode, a phase shift ϕ is added to minimize misalignment:

$$\phi = \arg \min_{\phi} \sum_i |t_i - (b_i + \phi)|$$

4. TSM Algorithms:

- **Phase Vocoder:** Computes the Short-Time Fourier Transform (STFT) of the audio, scales the time axis, and reconstructs the signal while preserving phase coherence.
- **WSOLA:** Splits the signal into overlapping frames, searches for optimal overlap positions based on waveform similarity, and performs overlap-add to stretch or compress the signal.

4 Project Deliverables

1. **Input Compatibility** – The system must successfully process two rhythmic signals (runner and music) whose base BPM differs by less than **10%**.
2. **Speed Ratio Range** – The Time-Scale Modification (TSM) algorithm shall operate stably for speed-up ratios between **0.8× and 1.5×**.
3. **Beat Alignment Accuracy** – When using beat-aware synchronization, the phase offset between runner steps and music beats shall not exceed **50 ms**.
4. **Processing Latency** – The total latency between receiving an updated runner rhythm and producing synchronized audio shall not exceed **2 seconds**.
5. **Audio Quality & Sampling Rate** – The output audio sampling rate must be at least **20 kHz**, with minimal distortion after TSM.

The first deliverable focuses on developing and testing the **Time-Scale Modification (TSM)** module, implemented using both **WSOLA** (time-domain) and the **Phase Vocoder** (frequency-domain) methods. Each will be applied to synthetic and real music clips with speed ratios between 0.8× and 1.5×. The processed durations will be compared to the target ratios to ensure deviations remain within $\pm 5\%$, while spectral similarity will be evaluated using **STFT correlation** between the original and modified audio. Expected results include audio samples, speed-ratio plots, and a comparison table summarizing ratio accuracy and audio quality.

The second deliverable extends the preprocessor for **beat alignment and synchronization**. It will estimate BPM and phase offset between runner and music signals, aligning beats using the calculated phase shift. **Cross-correlation analysis** will measure residual offsets for synthetic test cases with $\pm 10\%$ BPM differences, aiming for an average phase error below **50 ms**. Outputs will include aligned-beat visualizations and summary tables of phase errors.

The final deliverable integrates the **preprocessor and TSM** into a full prototype tested with synthetic and optional accelerometer data. The system's **end-to-end latency** will be measured using Python timing tools to ensure delays remain below **2 seconds**, and audio will be verified for correct ratio, alignment, and a sampling rate of at least **20 kHz**. The outcome will include a playback demo, latency and ratio stability results, and a final report confirming system performance meets specifications.

5 Project Schedule

A table with the main milestones (13-16 milestones). Please include a brief description of a milestone. The highlighted milestones are mandatory. (≤ 1 page)

#	Milestone	Description (2–3 lines)	Planned Date
1	Literature review & system planning	Research beat tracking, tempo estimation, and time-scaling methods. Define system architecture and datasets.	12.11.2025
2	BPM & beat detection (Librosa)	Implement BPM and beat extraction using Librosa's tempo/beat APIs. Test with synthetic clips.	26.11.2025
3	BPM-based preprocessing	Build preprocessing module that adjusts playback rate based on BPM difference between two sources.	10.12.2025
4	Phase-shift preprocessing	Extend preprocessing to estimate and correct phase	24.12.2025

		offset between tracks.	
5	Preprocessor live adaptation	Enable dynamic BPM tracking and automatic adaptation for changing runner/music inputs.	7.1.2026
6	Progress Presentation Submission	Present BPM and phase-alignment results, compare methods, and verify average residual phase offset < 50 ms.	Feb–Mar 2026
7	HPS (Harmonic–Percussive Separation) module	Implement HPS to isolate rhythmic and harmonic content before TSM.	20.3.2026
8	WSOLA implementation	Develop and test WSOLA algorithm for time-domain TSM across $0.8\times$ – $1.5\times$ ratios.	4.4.2026
9	Phase Vocoder implementation	Implement frequency-domain TSM and compare quality/ratio accuracy vs. WSOLA.	20.4.2026

10	Poster Submission and finishing the work	Prepare results, graphs, and comparisons for poster submission.	24.5.2026
11	System integration	Combine all modules into a unified real-time pipeline, including implementing the Live audio module.	14.6.2026
12	Full live adaptation	Add continuous real-time adaptation ensuring seamless tempo transitions.	25.6.2026
13	Performance & latency testing	Measure system latency, audio quality, and ratio stability under different conditions.	4.7.2026
14	Optional – hardware input	Integrate accelerometer or external BPM source for live testing.	18.7.2026
15	Final deliverables submission	Submit final report, demo, and all evaluation visuals.	Jul–Aug 2026